ПАРАЛЛЕЛЬНОЕ ПРОГРАММИРОВАНИЕ

Гибадуллин Р.Ф.
rfgibadullin@kai.ru

Презентация подготовлена на основе источника:
Албахари Д., Албахари Б. C# 7.0. Справочник. Полное описание языка.: Пер. с англ. – СпБ.: ООО «Альфа-книга», 2018. – С. 889-928. (См. главу 23. Параллельное программирование.)

Основные темы

В настоящей главе мы раскроем многопоточные API-интерфейсы и конструкции, направленные на использование преимуществ многоядерных процессоров:

- параллельный LINQ (Parallel LINQ), или PLINQ;
- класс `Parallel`;
- конструкции параллелизма задач;
- параллельные коллекции.

Все перечисленное было добавлено в версии .NET Framework 4.0 и вместе известно под (свободным) названием PFX (Parallel Framework). Класс `Parallel` и конструкции параллелизма задач называют *библиотекой параллельных задач* (Task Parallel Library – TPL).

Чтение главы требует знания основ, изложенных в главе 14, в частности блокирования, безопасности к потокам и класса `Task`.

Для чего нужна инфраструктура PFX?

На протяжении последнего десятилетия производители центральных процессоров (ЦП) перешли с одноядерной на многоядерную архитектуру. В результате создается дополнительная проблема для нас как программистов, поскольку однопоточный код не будет автоматически выполняться быстрее только по причине наличия дополнительных ядер.

Использовать в своих интересах множество ядер довольно легко в большинстве серверных приложений, где каждый поток может независимо обрабатывать отдельный клиентский запрос, но труднее в настольных приложениях, т.к. обычно это требует применения к коду с интенсивными вычислениями следующих действий.

1. *Разбиение* кода на небольшие части.
2. Выполнение частей кода параллельно через многопоточность.
3. *Объединение* результатов по мере их получения в безопасной к потокам и высокопроизводительной манере.

Концепции PFX

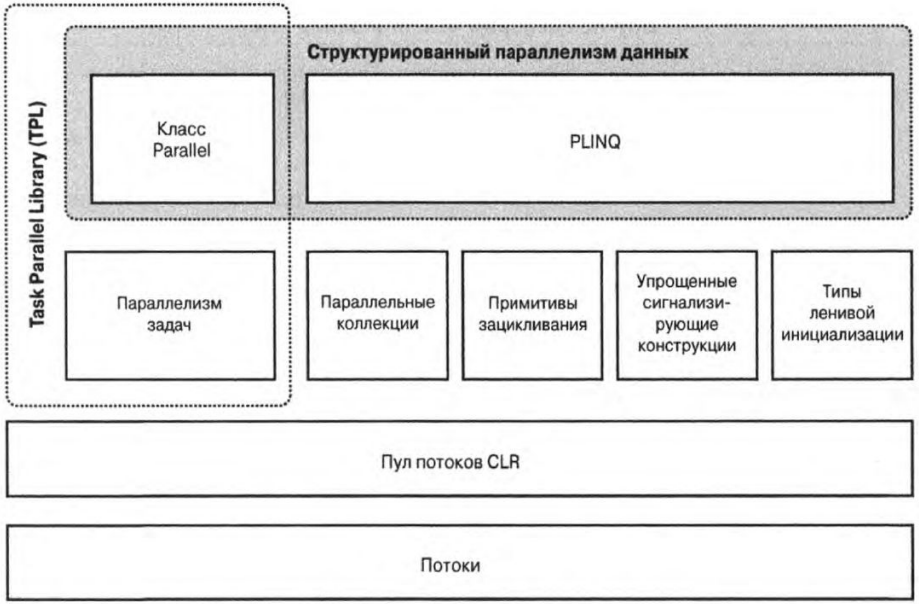
Существуют две стратегии разбиения работы между потоками: *параллелизм данных* и *параллелизм задач*.

Когда набор задач должен быть выполнен над множеством значений данных, мы можем распараллелить работу, заставив каждый поток выполнять (тот же самый) набор задач на подмножестве значений. Это называется *параллелизмом данных*, потому что мы распределяем *данные* между потоками. Напротив, при *параллелизме задач* мы распределяем *задачи*; другими словами, заставляем каждый поток выполнять свою задачу.

В общем случае параллелизм данных реализуется легче и масштабируется лучше для оборудования с высокой степенью параллелизма, т.к. он сокращает или устраняет разделяемые данные (и тем самым сводит к минимуму проблемы, связанные с состязаниями и безопасностью к потокам). Кроме того, параллелизм данных задействует тот факт, что значений данных часто имеется больше, чем дискретных задач, увеличивая в итоге потенциал параллелизма.

Параллелизм данных также способствует *структурированному параллелизму*, который означает, что параллельные единицы работы начинаются и завершаются в одном и том же месте внутри программы. В отличие от него параллелизм задач имеет тенденцию быть неструктурированным, т.е. параллельные единицы работы могут начинаться и завершаться в разных местах, разбросанных по программе. Структурированный параллелизм проще, менее подвержен ошибкам и позволяет поручить выполнение сложной работы по разбиению и координации потоков (и даже объединение результатов) библиотекам.

Компоненты PFX



	Разбивает работу	Объединяет результаты
PLINQ	Да	Да
Класс `Parallel`	Да	Нет
Параллелизм задач PFX	Нет	Нет

Когда необходимо использовать инфраструктуру PFX?

Основным сценарием использования PFX является *параллельное программирование*: привлечение множества процессорных ядер, чтобы ускорить выполнение интенсивного в плане вычислений кода.

Сложность с применением многоядерных процессоров обусловлена законом Амдала, который утверждает, что максимальное улучшение производительности от распараллеливания определяется той частью кода, которая должна выполняться последовательно. Например, если хотя бы две трети времени выполнения алгоритма поддаются распараллеливанию, то никогда не удастся превысить трехкратный выигрыш в производительности – даже при неограниченном числе ядер.

Таким образом, прежде чем продолжать, полезно проверить, что узкое место находится в распараллеливаемом коде. Также имеет смысл обдумать, *должен* ли ваш код действительно быть интенсивным в плане вычислений – часто простейшим и наиболее эффективным подходом будет оптимизация. Тем не менее, следует соблюдать компромисс, потому что некоторые технологии оптимизации могут затруднить распараллеливание кода.

Самый простой выигрыш получается с так называемыми *естественно параллельными* случаями – когда работа может быть легко разбита на задачи, которые сами по себе выполняются эффективно (здесь очень хорошо подходит структурированный параллелизм). Примеры включают многие задачи обработки изображений, метод трассировки лучей и прямолинейные подходы в математике или криптографии. Примером неестественной параллельной задачи может считаться реализация оптимизированной версии алгоритма быстрой сортировки – хороший результат требует некоторых размышлений и возможно неструктурированного параллелизма.

Parallel LINQ

Инфраструктура PLINQ автоматически распараллеливает локальные запросы LINQ. Преимущество PLINQ заключается в простоте использования, т.к. ответственность за выполнение работ по разбиению и объединению результатов перекладывается на .NET Framework.

Для применения PLINQ просто вызовите метод `AsParallel` на входной последовательности и затем продолжайте запрос LINQ обычным образом. Приведенный ниже запрос вычисляет простые числа между 3 и 100 000, обеспечивая полную загрузку всех ядер процессора на целевой машине:

```
// Вычислить простые числа с использованием простого (неоптимизированного) алгоритма.
IEnumerable<int> numbers = Enumerable.Range (3, 100000-3);
var parallelQuery =
  from n in numbers.AsParallel()
  where Enumerable.Range (2, (int) Math.Sqrt (n)).All (i => n % i > 0)
  select n;
int[] primes = parallelQuery.ToArray();
```

AsParallel представляет собой расширяющий метод в классе System.Linq.ParallelEnumerable. Он помещает входные данные в оболочку последовательности, основанной на ParallelQuery<TSource>, что вызывает привязку последующих операций запросов LINQ к альтернативному набору расширяющих методов, которые определены в классе ParallelEnumerable. Они предоставляют параллельные реализации для всех стандартных операций запросов. По существу они разбивают входную последовательность на порции, которые выполняются в разных потоках, и объединяют результаты снова в единственную выходную последовательность для дальнейшего потребления (рис. 23.2).

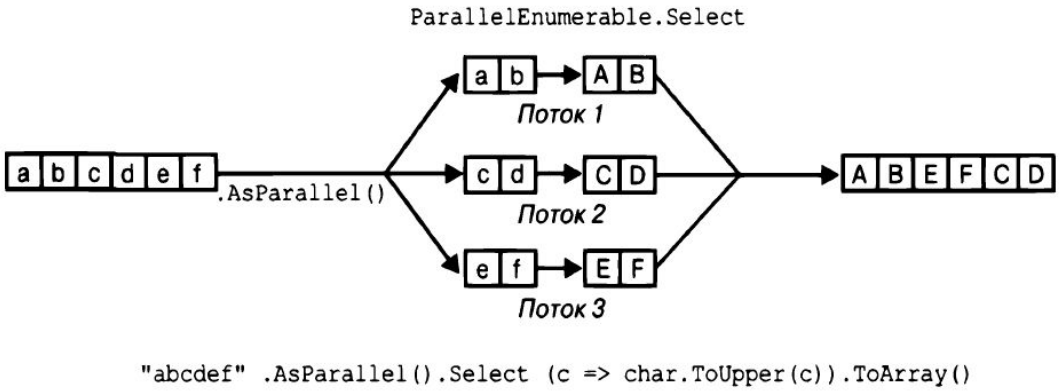


Рис. 23.2. Модель выполнения PLINQ

Не все операции запросов можно эффективно распараллеливать. Для операций, не поддающихся распараллеливанию (см. раздел “Ограничения PLINQ” далее в главе), PLINQ взамен реализует последовательное выполнение. Инфраструктура PLINQ может также оперировать последовательно, если ожидает, что накладные расходы от распараллеливания в действительности замедлят определенный запрос.

Инфраструктура PLINQ предназначена только для локальных коллекций: она не работает с LINQ to SQL или Entity Framework, потому что в таких ситуациях LINQ транслируется в код SQL, который затем выполняется на сервере баз данных. Тем не менее, PLINQ *можно* использовать для выполнения дополнительных локальных запросов в результирующих наборах, полученных из запросов к базам данных.

Если запрос PLINQ генерирует исключение, то оно повторно генерируется как объект `AggregateException`, свойство `InnerExceptions` которого содержит реальное исключение (или исключения). Дополнительные сведения можно найти в разделе “Работа с `AggregateException`” далее в главе.

Продвижение параллельного выполнения

Подобно обычным запросам LINQ запросы PLINQ оцениваются ленивым образом. Другими словами, выполнение будет инициировано, только когда начнется потребление результатов – как правило, посредством цикла `foreach` (хотя оно также может происходить через операцию преобразования, такую как `ToArray`, или операцию, которая возвращает одиночный элемент либо значение).

Тем не менее, при перечислении результатов выполнение продолжается несколько иначе, чем в случае обычного последовательного запроса. Последовательный запрос поддерживается полностью потребителем с применением модели с пассивным источником: каждый элемент из входной последовательности извлекается только тогда, когда он затребован потребителем. Параллельный запрос обычно использует независимые потоки для извлечения элементов из входной последовательности, причем с небольшим *упреждением*, до того момента, когда они понадобятся потребителю (почти как телесуфлер у дикторов новостей или буфер в проигрывателях компакт-дисков). Затем он обрабатывает элементы параллельно через цепочку запросов, удерживая результаты в небольшом буфере, чтобы они были готовы при затребовании потребителем. Если потребитель приостанавливает или прекращает перечисление до его завершения, обработчик запроса также приостанавливается или прекращает работу, чтобы не тратить впустую время ЦП или память.

Поведение буферизации PLINQ можно настраивать, вызывая метод `WithMergeOptions` после `AsParallel`. Стандартное значение `AutoBuffered` перечисления `ParallelMergeOptions` обычно дает наилучшие окончательные результаты. Значение `NotBuffered` отключает буфер и полезно в ситуации, когда результаты необходимо увидеть как можно скорее; значение `FullyBuffered` кеширует целый результирующий набор перед представлением его потребителю (подобным образом изначально работают операции `OrderBy` и `Reverse`, а также операции над элементами, операции агрегирования и операции преобразования).

PLINQ и упорядочивание

Побочный эффект от распараллеливания операций запросов заключается в том, что когда результаты объединены, они не обязательно находятся в том же самом порядке, в котором они были получены (см. рис. 23.2). Другими словами, обычная гарантия предохранения порядка LINQ для последовательностей больше не поддерживается.

Если нужно предохранение порядка, тогда после вызова `AsParallel` понадобится вызвать метод `AsOrdered`:

```
myCollection.AsParallel().AsOrdered()...
```

Вызов метода `AsOrdered` оказывает влияние на производительность, поскольку инфраструктура PLINQ должна отслеживать исходные позиции всех элементов.

Позже последствия от вызова `AsOrdered` в запросе можно отменить, вызвав метод `AsUnordered`: это вводит "точку случайного тасования", которая позволяет запросу выполняться более эффективно после ее прохождения. Таким образом, если необходимо предохранить упорядочение входной последовательности только для первых двух операций запросов, то можно поступить так:

```
inputSequence.AsParallel().AsOrdered()
  .QueryOperator1()
  .QueryOperator2()
  .AsUnordered()  // Начиная с этой точки, упорядочивание роли не играет
  .QueryOperator3()
  ...
```

Ограничения PLINQ

Существует несколько практических ограничений относительно того, что инфраструктура PLINQ способна распараллеливать. Ограничения могут быть ослаблены в последующих пакетах обновлений и версиях инфраструктуры .NET Framework.

Следующие операции запросов предотвращают распараллеливание запроса, если только исходные элементы не находятся в своих первоначальных индексных позициях:

- индексированные версии `Select`, `SelectMany` и `ElementAt`.

Большинство операций запросов изменяют индексные позиции элементов (включая операции, удаляющие элементы, такие как `Where`). Это означает, что если нужно использовать предшествующие операции, то они обычно должны располагаться в начале запроса.

Следующие операции запросов допускают распараллеливание, но применяют затратную стратегию разбиения, которая иногда может оказываться медленнее последовательной обработки:

- `Join`, `GroupBy`, `GroupJoin`, `Distinct`, `Union`, `Intersect` и `Except`.

Перегруженные версии операции `Aggregate`, принимающие начальное значение (в аргументе `seed`), в их стандартном виде не поддерживают возможность распараллеливания – в PLINQ для такой цели предлагаются специальные перегруженные версии (см. раздел “Оптимизация PLINQ” далее в главе).

Все остальные операции поддаются распараллеливанию, хотя их использование не гарантирует, что запрос будет распараллелен. Инфраструктура PLINQ может выполнять запрос последовательно, если ожидает, что накладные расходы от распараллеливания приведут к замедлению имеющегося конкретного запроса. Такое поведение можно переопределить и принудительно применять параллелизм, вызвав показанный ниже метод после `AsParallel`:

```
.WithExecutionMode (ParallelExecutionMode.ForceParallelism)
```

Пример: параллельная программа проверки орфографии

Предположим, что требуется написать программу проверки орфографии, которая выполняется быстро для очень больших документов за счет использования всех свободных процессорных ядер. Выразив алгоритм в виде запроса LINQ, мы легко можем его распараллелить.

Первый шаг предусматривает загрузку словаря английских слов в объект HashSet, чтобы обеспечить эффективный поиск:

```
if (!File.Exists ("WordLookup.txt"))  // Содержит около 150 000 слов
  new WebClient().DownloadFile (
    "http://www.albahari.com/ispell/allwords.txt", "WordLookup.txt");

var wordLookup = new HashSet<string> (
  File.ReadAllLines ("WordLookup.txt"),
  StringComparer.InvariantCultureIgnoreCase);
```

Затем мы будем применять полученное средство поиска слов для создания тестового "документа", содержащего массив из миллиона случайных слов. После построения массива мы внесем пару орфографических ошибок:

```
var random = new Random();
string[] wordList = wordLookup.ToArray();

string[] wordsToTest = Enumerable.Range (0, 1000000)
  .Select (i => wordList [random.Next (0, wordList.Length)])
  .ToArray();

wordsToTest [12345] = "woozsh";  // Внесение пары
wordsToTest [23456] = "wubsie";  // орфографических ошибок.
```

Теперь мы можем выполнить параллельную проверку орфографии, сверяя wordsToTest с wordLookup. PLINQ позволяет делать это очень просто:

```
var query = wordsToTest
  .AsParallel()
  .Select  ((word, index) => new IndexedWord { Word=word, Index=index })
  .Where   (iword => !wordLookup.Contains (iword.Word))
  .OrderBy (iword => iword.Index);

foreach (var mistake in query)
  Console.WriteLine (mistake.Word + " - index = " + mistake.Index);

// ВЫВОД:
// woozsh - index = 12345
// wubsie - index = 23456
```

`IndexedWord` — это специальная структура, которая определена следующим образом:

```
struct IndexedWord { public string Word; public int Index; }
```

Метод `wordLookup.Contains` в предикате придает запросу определенный “вес” и делает уместным его распараллеливание.

Мы могли бы слегка упростить запрос за счет использования анонимного типа вместо структуры `IndexedWord`. Однако это привело бы к снижению производительности, т.к. анонимные типы (будучи классами, а потому ссылочными типами) привносят накладные расходы на выделение памяти в куче и последующую сборку мусора.

Разница может оказаться недостаточной, чтобы иметь значение в последовательных запросах, но в случае параллельных запросов весьма выгодно отдавать предпочтение выделению памяти в стеке. Причина в том, что выделение памяти в стеке хорошо поддается распараллеливанию (поскольку каждый поток имеет собственный стек), в то время как в противном случае все потоки должны состязаться за одну и ту же кучу, управляемую единственным диспетчером памяти и сборщиком мусора.

Использование ThreadLocal<T>

Давайте расширим наш пример, распараллелив само создание случайного тестового списка слов. Мы структурировали его как запрос LINQ, так что все должно быть легко. Вот последовательная версия:

```
string[] wordsToTest = Enumerable.Range (0, 1000000)
  .Select (i => wordList [random.Next (0, wordList.Length)])
  .ToArray();
```

К сожалению, вызов метода `random.Next` небезопасен в отношении потоков, поэтому работа не сводится к простому добавлению в запрос вызова `AsParallel`. Потенциальным решением может быть написание функции, помещающей вызов `random.Next` внутрь блокировки, но это ограничило бы параллелизм. Более удачный вариант предусматривает применение класса `ThreadLocal<Random>` (см. раздел "Локальное хранилище потока" в главе 22) с целью создания отдельного объекта `Random` для каждого потока. Тогда распараллелить запрос можно следующим образом:

```
var localRandom = new ThreadLocal<Random>
 ( () => new Random (Guid.NewGuid().GetHashCode()) );

string[] wordsToTest = Enumerable.Range (0, 1000000).AsParallel()
  .Select (i => wordList [localRandom.Value.Next (0, wordList.Length)])
  .ToArray();
```

В нашей фабричной функции для создания объекта `Random` мы передаем хеш-код `Guid`, гарантируя тем самым, что даже если два объекта `Random` создаются в рамках короткого промежутка времени, то они все равно будут выдавать отличающиеся последовательности случайных чисел.

Когда необходимо использовать PLINQ?

Довольно заманчиво поискать в существующих приложениях запросы LINQ и поэкспериментировать с их распараллеливанием. Однако обычно это непродуктивно, т.к. большинство задач, для которых LINQ является очевидным лучшим решением, выполняются очень быстро, а потому не выигрывают от распараллеливания. Более удачный подход предполагает поиск узких мест, интенсивно использующих ЦП, и выяснение, могут ли они быть выражены в виде запроса LINQ. (Приятный побочный эффект от такой реструктуризации состоит в том, что LINQ обычно делает код более кратким и читабельным.)

Инфраструктура PLINQ хорошо подходит для естественно параллельных задач. Однако она может быть плохим выбором для обработки изображений, потому что объединение миллионов пикселей в выходную последовательность создаст узкое место. Взамен пиксели лучше записывать прямо в массив или блок неуправляемой памяти и применять класс `Parallel` либо параллелизм задач для управления многопоточностью. (Тем не менее, объединение результатов можно аннулировать с использованием `ForAll` – мы обсудим данную тему в разделе “Оптимизация PLINQ” далее в главе. Поступать так имеет смысл, если алгоритм обработки изображений естественным образом приспосабливается к LINQ.)

Функциональная чистота

Давайте расширим наш пример, распараллелив само создание случайного тестового списка слов. Мы структурировали его как запрос LINQ, так что все должно быть легко. Вот последовательная версия:

```
string[] wordsToTest = Enumerable.Range (0, 1000000)
  .Select (i => wordList [random.Next (0, wordList.Length)])
  .ToArray();
```

К сожалению, вызов метода random.Next небезопасен в отношении потоков, поэтому работа не сводится к простому добавлению в запрос вызова AsParallel. Потенциальным решением может быть написание функции, помещающей вызов random.Next внутрь блокировки, но это ограничило бы параллелизм. Более удачный вариант предусматривает применение класса ThreadLocal<Random> (см. раздел "Локальное хранилище потока" в главе 22) с целью создания отдельного объекта Random для каждого потока. Тогда распараллелить запрос можно следующим образом:

```
var localRandom = new ThreadLocal<Random>
 ( () => new Random (Guid.NewGuid().GetHashCode()) );

string[] wordsToTest = Enumerable.Range (0, 1000000).AsParallel()
  .Select (i => wordList [localRandom.Value.Next (0, wordList.Length)])
  .ToArray();
```

В нашей фабричной функции для создания объекта Random мы передаем хеш-код Guid, гарантируя тем самым, что даже если два объекта Random создаются в рамках короткого промежутка времени, то они все равно будут выдавать отличающиеся последовательности случайных чисел.

Мы могли бы сделать инкрементирование переменной `i` безопасным к потокам за счет применения блокировок, но все еще останется проблема того, что `i` не обязательно будет соответствовать позиции входного элемента. И добавление `AsOrdered` в запрос не решит последнюю проблему, т.к. метод `AsOrdered` гарантирует лишь то, что элементы выводятся в порядке, согласованном с порядком, который бы они имели при последовательной обработке — он не осуществляет действительную их *обработку* последовательным образом.

Взамен данный запрос должен быть переписан с использованием индексированной версии `Select`:

```
var query = Enumerable.Range(0,999).AsParallel().Select ((n, i) => n * i);
```

Для достижения лучшей производительности любые методы, вызываемые из операций запросов, должны быть безопасными к потокам, не производя запись в поля или свойства (не давать побочные эффекты, т.е. быть *функционально чистыми*). Если они являются безопасными в отношении потоков благодаря блокированию, тогда потенциал параллелизма запроса будет ограничен продолжительностью действия блокировки, деленной на общее время, которое занимает выполнение данной функции.

Давайте расширим наш пример, распараллелив само создание случайного тестового списка слов. Мы структурировали его как запрос LINQ, так что все должно быть легко. Вот последовательная версия:

```
string[] wordsToTest = Enumerable.Range (0, 1000000)
  .Select (i => wordList [random.Next (0, wordList.Length)])
  .ToArray();
```

К сожалению, вызов метода `random.Next` небезопасен в отношении потоков, поэтому работа не сводится к простому добавлению в запрос вызова `AsParallel`. Потенциальным решением может быть написание функции, помещающей вызов `random.Next` внутрь блокировки, но это ограничило бы параллелизм. Более удачный вариант предусматривает применение класса `ThreadLocal<Random>` (см. раздел "Локальное хранилище потока" в главе 22) с целью создания отдельного объекта `Random` для каждого потока. Тогда распараллелить запрос можно следующим образом:

```
var localRandom = new ThreadLocal<Random>
 ( () => new Random (Guid.NewGuid().GetHashCode()) );

string[] wordsToTest = Enumerable.Range (0, 1000000).AsParallel()
  .Select (i => wordList [localRandom.Value.Next (0, wordList.Length)])
  .ToArray();
```

В нашей фабричной функции для создания объекта `Random` мы передаем хеш-код `Guid`, гарантируя тем самым, что даже если два объекта `Random` создаются в рамках короткого промежутка времени, то они все равно будут выдавать отличающиеся последовательности случайных чисел.

Установка степени параллелизма

По умолчанию PLINQ выбирает оптимальную степень параллелизма для задействованного процессора. Ее можно переопределить, вызвав метод `WithDegreeOfParallelism` после `AsParallel`:

```
...AsParallel().WithDegreeOfPallelism(4)...
```

Изменение степени параллелизма

Метод `WithDegreeOfParallelism` можно вызывать только один раз внутри запроса PLINQ. Если его необходимо вызвать снова, то потребуется принудительно инициировать слияние и повторное разбиение запроса, еще раз вызвав метод `AsParallel` внутри запроса:

```
"The Quick Brown Fox"
  .AsParallel().WithDegreeOfParallelism (2)
  .Where (c => !char.IsWhiteSpace (c))
  .AsParallel().WithDegreeOfParallelism (3) //Инициировать слияние и разбиение
  .Select (c => char.ToUpper (c))
```

Отмена

Отменить запрос, который заканчивается операцией преобразования, операцией над элементами или операцией агрегирования, можно из другого потока через признак отмены (см. раздел “Отмена” в главе 14). Чтобы вставить такой признак, необходимо после вызова `AsParallel` вызвать метод `WithCancellation`, передав ему свойство `Token` объекта `CancellationTokenSource`. Затем другой поток может вызвать метод `Cancel` на источнике признака, что приведет к генерации исключения `OperationCanceledException` в потребителе запроса:

Инфраструктура PLINQ не прекращает потоки вытесняющим образом из-за связанной с этим опасности (см. раздел “`Interrupt` и `Abort`” в главе 22). Взамен при инициировании отмены она ожидает завершения каждого рабочего потока со своим текущим элементом перед тем, как закончить запрос. Это означает, что любые внешние методы, которые вызывает запрос, будут выполняться до полного завершения.

```
IEnumerable<int> million = Enumerable.Range (3, 1000000);
var cancelSource = new CancellationTokenSource();
var primeNumberQuery =
  from n in million.AsParallel().WithCancellation (cancelSource.Token)
  where Enumerable.Range (2, (int) Math.Sqrt (n)).All (i => n % i > 0)
  select n;
new Thread (() => {
                    Thread.Sleep (100);       // Отменить запрос по
                    cancelSource.Cancel();    // прошествии 100 мс.
                  }
           ).Start();
try
{
  // Начать выполнение запроса:
  int[] primes = primeNumberQuery.ToArray();
  // Мы никогда не попадем сюда, потому что другой поток инициирует отмену.
}
catch (OperationCanceledException)
{
  Console.WriteLine ("Query canceled");      // Запрос отменен
}
```

Оптимизация PLINQ

Оптимизация на выходной стороне

Одно из преимуществ инфраструктуры PLINQ связано с тем, что она удобно объединяет результаты распараллеленной работы в единую выходную последовательность. Однако иногда все, что в итоге делается с такой последовательностью – выполнение некоторой функции над каждым элементом:

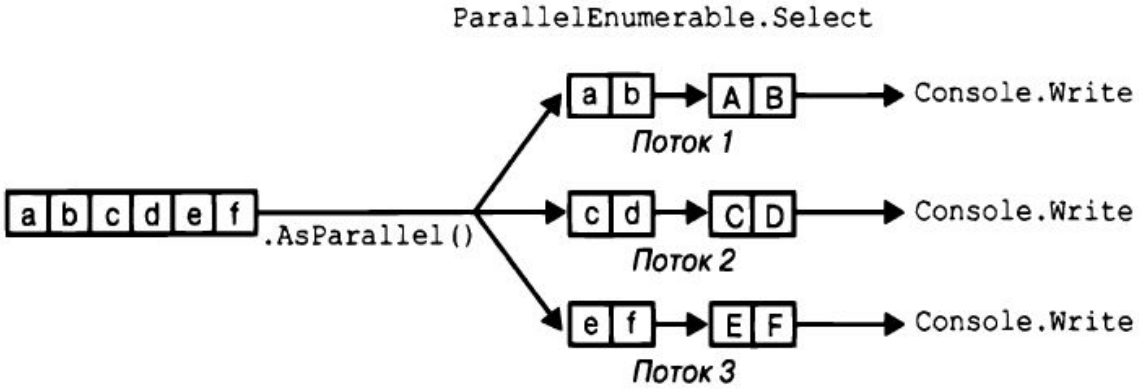
```
foreach (int n in parallelQuery)
  DoSomething (n);
```

В таком случае, если порядок обработки элементов не волнует, тогда эффективность можно улучшить с помощью метода `ForAll` из PLINQ.

Метод `ForAll` запускает делегат для каждого выходного элемента `ParallelQuery`. Он проникает прямо внутрь PLINQ, обходя шаги объединения и перечисления результатов. Ниже приведен простейший пример:

```
"abcdef".AsParallel().Select (c => char.ToUpper(c)).ForAll (Console.Write);
```

Процесс продемонстрирован на рис. 23.3.




```
"abcdef" .AsParallel().Select (c => char.ToUpper(c)).ForAll (Console.Write)
```

Рис. 23.3. *Метод* `ForAll` *из PLINQ*

Объединение и перечисление результатов — не массовая затратная операция, поэтому оптимизация с помощью `ForAll` дает наибольшую выгоду при наличии большого количества быстро обрабатываемых входных элементов.